Refs & useRef: Beyond DOM Access

10 min read Updated 26 Mar 2026 Topic 9 of 62

1.9 — Refs & useRef: Beyond DOM Access

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is useRef and what is it used for?"	🟢🟡	★★★★★
"What is the difference between useRef and useState?"	🟡	★★★★☆
"What is forwardRef?"	🟡	★★★★☆
"How do you access a DOM element in React?"	🟢	★★★★☆
"Does changing a ref trigger a re-render?"	🟡	★★★★☆
"What is useImperativeHandle?"	🔴	★★☆☆☆

What is useRef?

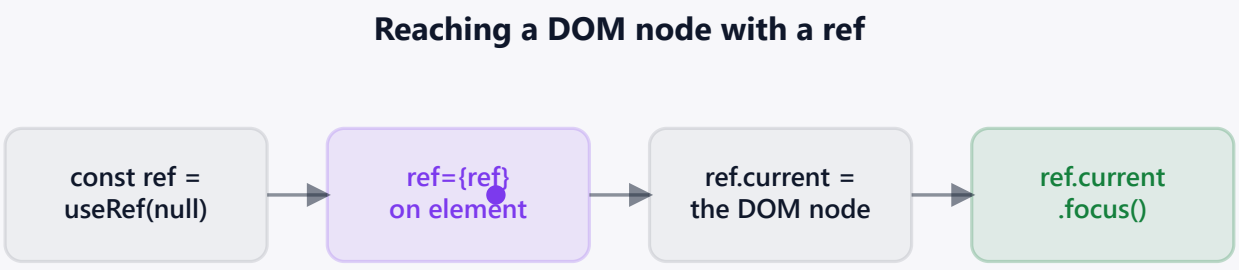
`useRef` returns a mutable object with a single property: `.current`. That's it. It's a box that can hold any value, and that value persists across renders without causing re-renders when it changes.

JSX Copy

```jsx
const myRef = useRef(initialValue);
// myRef is { current: initialValue }
// You can read and write myRef.current freely
```

People usually learn about refs for DOM access, but that's only half the story. Refs have two completely different use cases.

Use Case 1: Accessing DOM Elements



After render, React fills ref.current with the real DOM element — then you can call imperative APIs like .focus().

This is the most visible use of refs. You create a ref, attach it to a JSX element, and React populates `.current` with the actual DOM node after the component mounts.

JSX Copy

```jsx
function SearchBar() {
  const inputRef = useRef(null);

  const handleButtonClick = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <input ref={inputRef} placeholder="Search ..." />
      <button onClick={handleButtonClick}>Focus Search</button>
    </div>
  );
}
```

Scroll to an Element

JSX Copy

```jsx
function LongPage() {
  const bottomRef = useRef(null);

  const scrollToBottom = () => {
    bottomRef.current.scrollIntoView({ behavior: 'smooth' });
  };

  return (
    <div>
      <button onClick={scrollToBottom}>Go to bottom</button>
      {/* ...lots of content... */}
      <div ref={bottomRef}>Bottom of page</div>
    </div>
  );
}
```

Media Playback Control

```jsx
function AudioPlayer({ src }) {
  const audioRef = useRef(null);

  return (
    <div>
      <audio ref={audioRef} src={src} />
      <button onClick={() => audioRef.current.play()}>Play</button>
      <button onClick={() => audioRef.current.pause()}>Pause</button>
    </div>
  );
}
```

Measuring DOM Dimensions

JSX Copy

```jsx
function MeasuredBox() {
  const boxRef = useRef(null);
  const [dimensions, setDimensions] = useState({ width: 0, height: 0 });

  useEffect(() => {
    if (boxRef.current) {
      const { width, height } = boxRef.current.getBoundingClientRect();
      setDimensions({ width, height });
    }
  }, []);

  return (
    <div ref={boxRef} style={{ padding: 20, border: '1px solid gray' }}>
      This box is {Math.round(dimensions.width)}px wide
      and {Math.round(dimensions.height)}px tall
    </div>
  );
}
```

Use Case 2: Storing Mutable Values Without Re-rendering



Writing to ref.current mutates in place — perfect for data the render output does not depend on.

This is the use case most beginners miss, and it comes up in interviews frequently.

Unlike `useState`, updating `ref.current` does **not** trigger a re-render. This makes refs perfect for storing values you need to track but don't want to display.

Tracking Previous State

JSX Copy

```jsx
function Counter() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef(0);

  useEffect(() => {
    prevCountRef.current = count;  // Update after each render
  });

  return (
    <div>
      <p>Current: {count}</p>
      <p>Previous: {prevCountRef.current}</p>
      <button onClick={() => setCount(c => c + 1)}>+</button>
    </div>
  );
}
```

Storing Timer IDs

```jsx
function Stopwatch() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null);

  const start = () => {
    if (intervalRef.current) return;  // Prevent multiple intervals
    intervalRef.current = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);
  };

  const stop = () => {
    clearInterval(intervalRef.current);
    intervalRef.current = null;
  };

  const reset = () => {
    stop();
    setSeconds(0);
  };

  useEffect(() => {
    return () => clearInterval(intervalRef.current);  // Cleanup on unmoun
  }, []);

  return (
    <div>
      <p>{seconds} seconds</p>
      <button onClick={start}>Start</button>
      <button onClick={stop}>Stop</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}
```

If you stored the interval ID in state, every `setIntervalId` call would trigger an unnecessary re-render. The interval ID has nothing to do with the UI. Ref is the right tool here.

JSX Copy

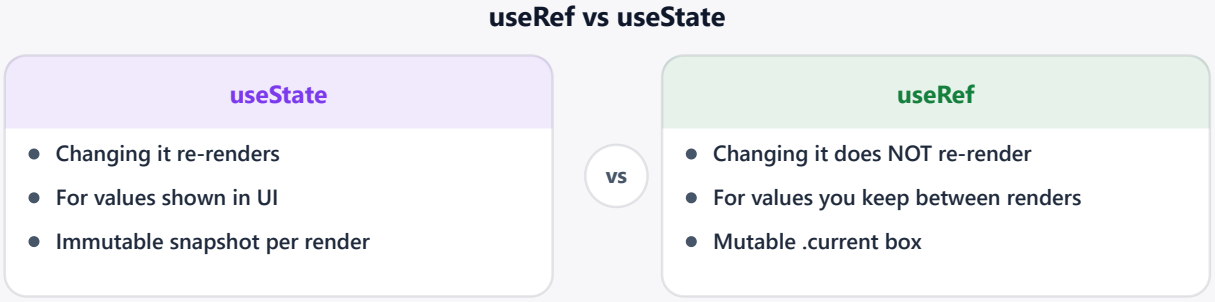
```jsx
function RenderCounter() {
  const renderCount = useRef(0);
  renderCount.current += 1;

  return <p>This component has rendered {renderCount.current} times</p>;
}
```

If you tried this with `useState`, you'd create an infinite loop: setting state triggers a re-render, which increments the count, which sets state, which triggers a re-render...

---

useRef vs useState



Use state for what the user sees; use a ref for what you need to remember but not display.

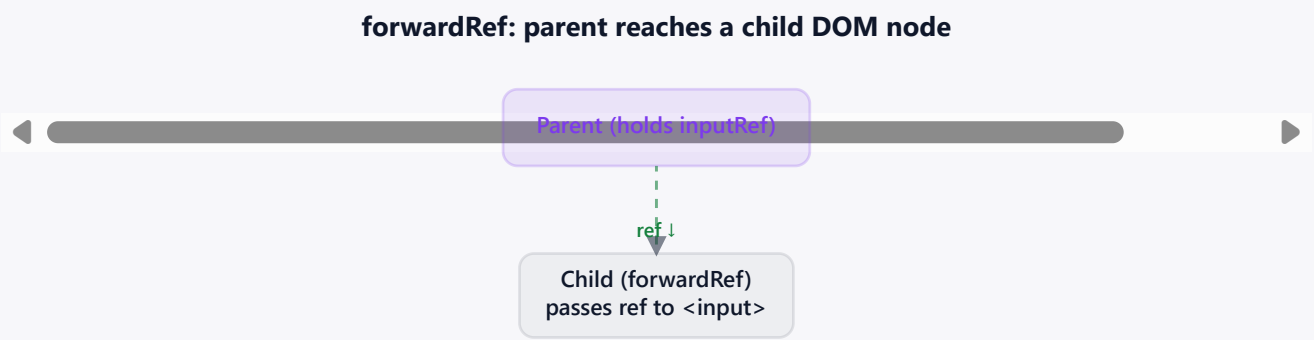
	`useRef`	`useState`
Triggers re-render on update?	No	Yes

persists across renders?	Yes	Yes
When to use	DOM access, timer IDs, previous values, render counts	Data that needs to appe
Update syntax	`ref.current = newValue`	`setState(newValue)`
Returns	`{ current: value }`	`[value, setter]`

The mental model: if changing a value should update what the user sees on screen, use state. If it should NOT affect the screen, use a ref.

---

forwardRef: Passing Refs to Child Components



By default refs do not pass through components. forwardRef lets a parent attach its ref to a DOM node inside a child.

By default, you can't pass a `ref` to a function component. React doesn't forward it automatically because `ref` isn't a regular prop.

```jsx
// ❌ This doesn't work. ref won't reach the input.
function CustomInput(props) {
  return <input className="custom" {...props} />;
}

function Parent() {
  const inputRef = useRef();
  return <CustomInput ref={inputRef} />;  // Warning: ref is not a prop
}
```

`forwardRef` solves this. It lets a component receive a ref from its parent and attach it to an inner DOM element.

JSX Copy

```jsx
// ✅ forwardRef passes the ref through to the inner element
const CustomInput = forwardRef((props, ref) => {
  return <input ref={ref} className="custom" {...props} />;
});

function Parent() {
  const inputRef = useRef();

  return (
    <div>
      <CustomInput ref={inputRef} placeholder="Type here..." />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </div>
  );
}
```

The parent creates a ref, passes it to `CustomInput`, and `forwardRef` makes it available as the second argument. The child attaches it to the actual `<input>` element. Now the parent can call `.focus()` on the real DOM node inside the child.

When You Need forwardRef

access.

Component libraries like Material UI and Chakra use forwardRef extensively for this reason.

---

useImperativeHandle (Senior Level)

Sometimes you don't want to expose the entire DOM node to the parent. You want to give the parent a limited, custom API. That's what `useImperativeHandle` does.

JSX Copy

```jsx
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef();

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => { inputRef.current.value = ''; },
    getValue: () => inputRef.current.value,
  }));

  return <input ref={inputRef} {...props} />;
});

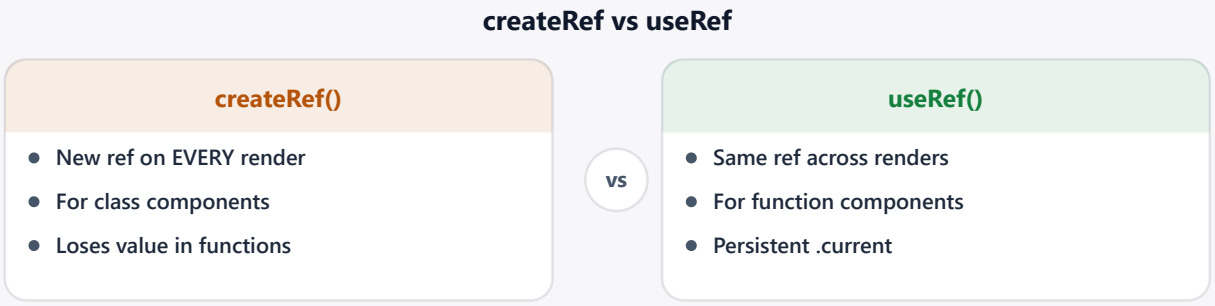
function Parent() {
  const fancyRef = useRef();

  return (
    <div>
      <FancyInput ref={fancyRef} />
      <button onClick={() => fancyRef.current.focus()}>Focus</button>
      <button onClick={() => fancyRef.current.clear()}>Clear</button>
      <button onClick={() => alert(fancyRef.current.getValue())}>Get Value
    </div>
  );
}
```

Instead of giving the parent the raw `<input>` DOM node (which has hundreds of properties and methods), you expose just three: `focus`, `clear`, and `getValue`. This is cleaner and prevents the parent from doing things you didn't intend.

---

createRef vs useRef



In function components always use useRef — createRef would reset on every render.

This comes up in interviews when discussing class vs functional components.

	`useRef`	`createRef`
Used in	Functional components	Class components
Returns same object across renders?	Yes (persistent)	No (creates a new ref each render)
Common usage	Standard in modern React	Legacy class components

JSX Copy

```jsx
// Functional component: useRef
function MyComponent() {
  const ref = useRef(null);  // Same ref object every render
```

```jsx
// Class component: createRef
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.ref = React.createRef();  // Created once in constructor
  }
  render() {
    return <input ref={this.ref} />;
  }
}
```

If you use `createRef` inside a functional component's body (not in a constructor), it creates a new ref on every render and the previous value is lost. That's why `useRef` exists.

Common Mistakes

Mistake 1: Reading ref.current during render. The ref is `null` during the first render because the DOM hasn't been created yet. Always access refs inside `useEffect` or event handlers.

JSX Copy

```jsx
// ❌ ref.current is null here
function Bad() {
  const ref = useRef(null);
  console.log(ref.current.offsetWidth);  // Crash! null.offsetWidth
  return <div ref={ref}>Hello</div>;
}

// ✅ Access ref after mount
function Good() {
  const ref = useRef(null);
  useEffect(() => {
    console.log(ref.current.offsetWidth);  // Works, DOM exists now
  }, []);
```

Mistake 2: Expecting ref changes to update the UI. If you store a value in `ref.current` and display it in JSX, the screen won't update when you change it. Use state for anything visible.

Mistake 3: Using refs when state would be simpler. Beginners sometimes reach for refs to avoid re-renders. But if the value needs to appear in the UI, you need a re-render, and state is the right tool. Don't optimize for a problem you don't have.

🔥 Output Question

JSX Copy

```jsx
function App() {
  const countRef = useRef(0);

  const handleClick = () => {
    countRef.current += 1;
    console.log('Ref value:', countRef.current);
  };

  return (
    <div>
      <p>Count: {countRef.current}</p>
      <button onClick={handleClick}>Increment</button>
    </div>
  );
}
```

What happens when you click the button 3 times?

The console logs `1`, `2`, `3` correctly. But the screen always shows `Count: 0`. The paragraph never updates because changing `ref.current` does not trigger a re-render. The ref value is correct internally, but React has no reason to re-paint the component.

Scenario: "Build a click-outside-to-close dropdown"

This is a classic machine coding question that tests refs, event listeners, and cleanup.

JSX

Copy

```jsx
function Dropdown({ trigger, children }) {
  const [isOpen, setIsOpen] = useState(false);
  const dropdownRef = useRef(null);

  useEffect(() => {
    const handleClickOutside = (event) => {
      if (dropdownRef.current && !dropdownRef.current.contains(event.targe
        setIsOpen(false);
      }
    };

    if (isOpen) {
      document.addEventListener('mousedown', handleClickOutside);
    }

    return () => {
      document.removeEventListener('mousedown', handleClickOutside);
    };
  }, [isOpen]);

  return (
    <div ref={dropdownRef} style={{ position: 'relative', display: 'inline
      <button onClick={() => setIsOpen(prev => !prev)}>{trigger}</button>
      {isOpen && (
        <div style={{
          position: 'absolute',
          top: '100%',
          left: 0,
          border: '1px solid #ccc',
          background: 'white',
          padding: 8,
          zIndex: 10,
        }}>
          {children}
        </div>
      )}
    </div>
  );
```

```
// Usage
<Dropdown trigger="Options">
  <p>Option 1</p>
  <p>Option 2</p>
  <p>Option 3</p>
</Dropdown>
```

This demonstrates: ref for DOM containment check, useEffect for adding/removing global event listeners, cleanup to prevent memory leaks, and the children prop for flexible content.

🗣️ Interview Answer

> "useRef returns a mutable object with a `.current` property that persists across renders. It has two main uses. First, accessing DOM elements directly, like focusing an input or measuring dimensions. Second, storing mutable values that don't need to trigger re-renders, like timer IDs or previous state values. The key difference from useState is that updating ref.current does not cause a re-render. forwardRef is needed when you want to pass a ref from a parent to a child function component, typically for reusable input or modal components."

✅ Key Takeaways

- `useRef` creates a persistent mutable container: `{ current: value }`.
- **Two use cases:** DOM access and storing values without triggering re-renders.
- Updating `ref.current` does **not** cause a re-render. Use state for anything visible.
- Access refs in **useEffect or event handlers**, never during render (it's null on first render).

- `useImperativeHandle` lets you expose a custom, limited API to the parent instead of the raw DOM node.
- `createRef` is for class components. `useRef` is for functional components. Don't mix them.
- Common real-world uses: focus management, scroll control, media playback, click-outside detection, timer management, tracking previous values, render counting.

QUICK CHECK — no marks, just for you.

1. A component stores a counter in countRef.current and renders {countRef.current} inside a paragraph. Clicking a button increments countRef.current. After three clicks, what does the user see on screen?

- ◯ Count: 2, because the paragraph lags one render behind the ref
- ◯ Count: 0, because mutating ref.current does not trigger a re-render
- ◯ Count: 3, because the ref object is mutable and React reads it on every paint
- ◯ Nothing renders, because reading ref.current during render throws an error

Check answer

2. You wrote a reusable CustomInput function component that renders an <input>. A parent does <CustomInput ref={inputRef} /> and then calls inputRef.current.focus(), but it fails. According to the lesson, what does forwardRef change?

- ◯ It delays ref assignment until after useEffect runs, so .current is no longer null
- ◯ It lets the component receive the parent's ref as a second argument so it can attach it to the inner DOM element
- ◯ It makes the component re-render whenever the parent's ref object changes

It converts ref into an ordinary prop that the component reads from props.ref

Check answer

3. A developer calls React.createRef() directly in the body of a function component instead of useRef(). What goes wrong?

- ○ The ref works identically to useRef, but the initial value must be null
- ○ createRef throws, because it may only be called inside a class constructor
- ○ A new ref object is created on every render, so the previously stored value is lost
- ○ The ref persists but never receives the DOM node, since only useRef is wired to JSX

Check answer

‹ Previous
Event Handling in React

Next ›
Error Boundaries & Handling Errors Gracefully

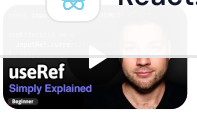
LEARN MORE

ARTICLES & DOCS

- Referencing Values with Refs
  react.dev
- Manipulating the DOM with Refs
  react.dev
- useRef — API reference
  react.dev

VIDEOS

React.js Interview Kit



useRef — simply explained

YouTube



useRef vs useState — compared

YouTube



The useRef hook in React

YouTube

Hand-picked references for this lesson.